

PROGRAMARE AVANSATĂ PE OBIECTE ÎN JAVA

Conf.univ.dr. Radu BORIGA





1. **Introducere în limbajul Java**
2. **Clase și obiecte. Extinderea claselor. Polimorfism**
3. **Tablouri. Șiruri de caractere**
4. **Clase abstracte. Interfețe**
5. **Excepții**
6. **Fluxuri de intrare/ieșire**
7. **Colecții de date**
8. **Lambda expresii + stream-uri**
9. **Design patterns**
10. **Fire de executare**
11. **Socket-uri**
12. **Lucrul cu baze de date (JDBC)**



Bibliografie

- Cay Horstmann, *Core Java* (vol. I–II), 14th edition, Pearson Education, 2025
- Victor Grazi, Jeanne Boyarsky, *Real-World Java*, John Wiley & Sons, 2025
- Raul Gabriel Urma, *Modern Java in action*, 2nd edition, Manning Publications, 2021
- Joshua Bloch, *Effective Java*, 3rd edition, Addison-Wesley Professional, 2018
- Bruce Eckel, *Thinking in Java*, 4th edition, Pearson Education, 2006

➤ Tutoriale:

- <https://dev.java/>
- <https://www.vogella.com/tutorials/java.html>
- <https://www.baeldung.com/>



- 50% → Proiect (minim nota 5 **fără rotunjire**)
- 50% → Examen (minim nota 5 **fără rotunjire**)

Tematica cursului 1



- › **Prezentarea generală a platformei Java**
- › **Structura unui program**
- › **Tipuri de date și operatori**
- › **Literali**
- › **Instrucțiuni**
- › **Pachete de clase**
- › **Operații de citire/scriere**

Scurt istoric al limbajului Java



3 Billion Devices Run Java

ATMs, Smartcards, POS Terminals, Blu-ray Players, Set Top Boxes, Multifunction Printers, PCs, Servers, Routers, Switches, Parking Meters, Smart Meters, Lottery Systems, Airplane Systems, IoT Gateways, Programmable Logic Controllers, Optical Sensors, Wireless M2M Modules, Access Control Systems, Medical Devices, Building Controls, Automobiles...

 Java™ | #1 Development Platform

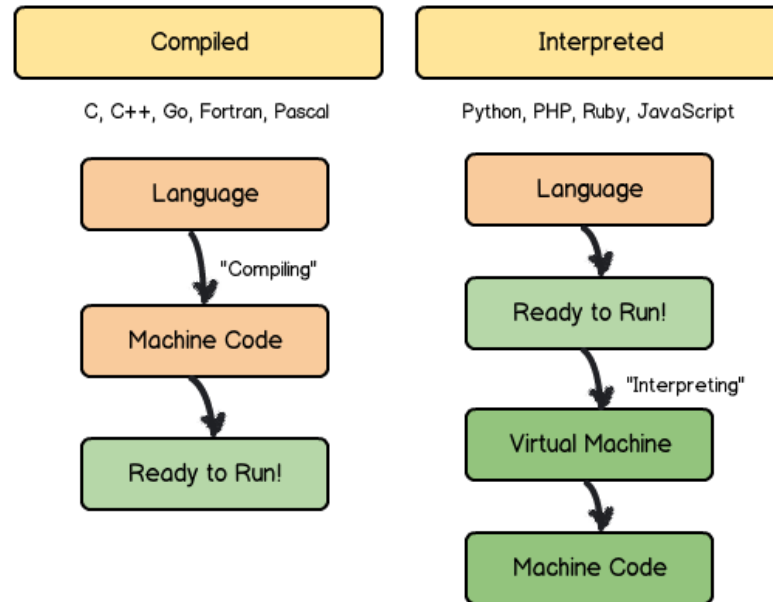
ORACLE®

- În anul 1991 firma Sun Microsystems finanțează proiectul Green, coordonat de James Gosling.
- Specificațiile noului limbaj, inițial denumit OAK, iar apoi Java 1.0 sunt finalizate în anul 1995.
- În anul 1995 compania Sun Microsystems vinde licența firmelor IBM, Microsoft, Adobe și Netscape.
- În 2009 Sun Microsystems este cumpărată de Oracle.
- Java 1.x (1996-2004): JDBC, JIT, AWT, Swing, NIO, Collections
- Java 5 (2004): tipuri generice, enumerări, programare concurentă
- Java 8 (2014): lambda expresii și programare funcțională
- Java 16 – 19 (2021-2023): records, sealed class, pattern matching pentru switch
- Java 21 (19 septembrie 2023): fire de executare virtuale
- Java 25 (16 septembrie 2025): programare concurentă structurată

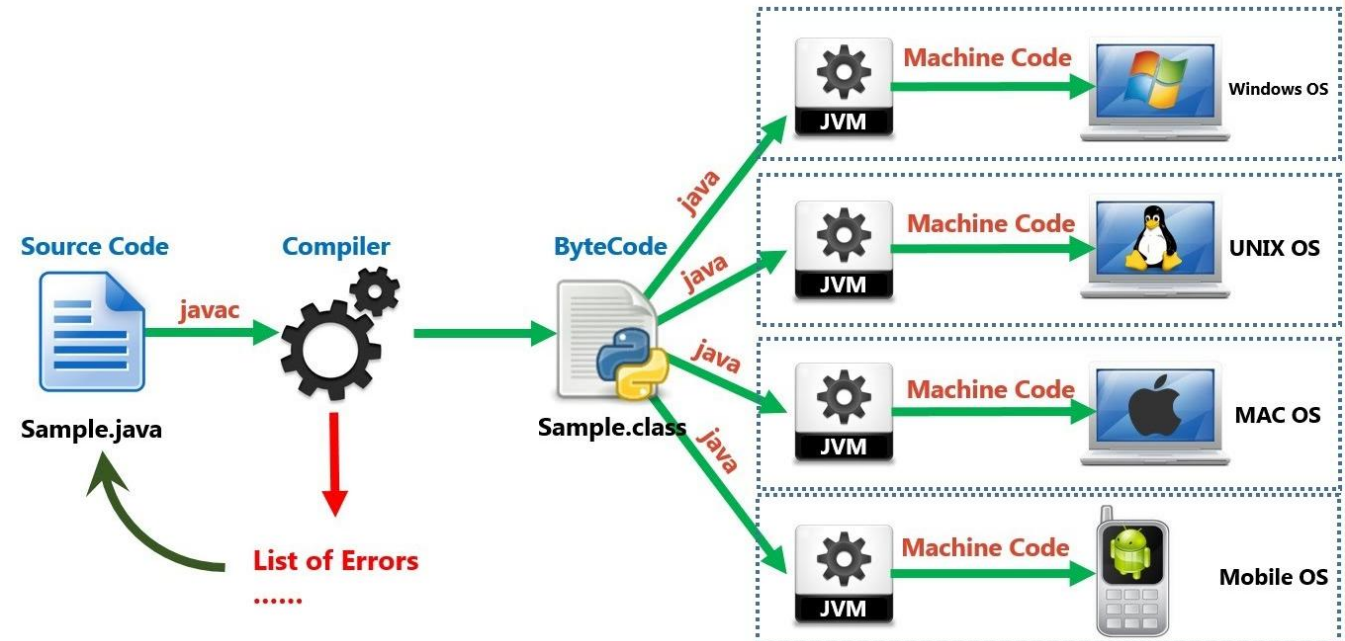


Caracteristici

› Limbaj compilat și interpretat



› Write Once, Run Anywhere



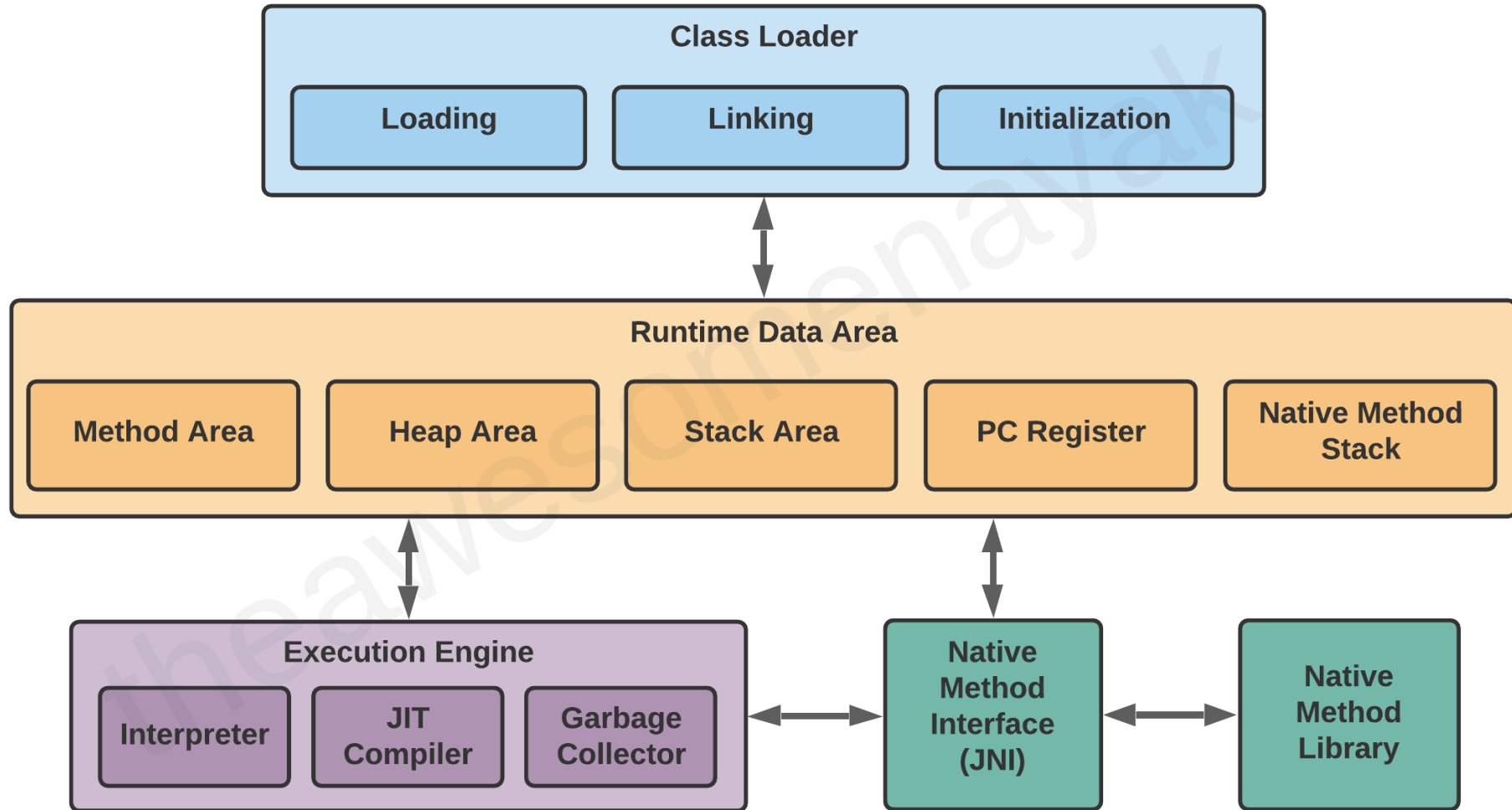


Caracteristici

- › Bytecode-ul reprezintă un set de instrucțiuni specifice JVM

C++/Java	X86 ASM	Java bytecode (mnemonics)	Java bytecode (hexadecimal)
int add(int a, int b) { return a+b; }	mov eax, byte[ebp-4]	iload_0	0x1A
	mov edx, byte[ebp-8]	iload_1	0x1B
	add eax, edx	iadd	0x60
	ret	ireturn	0xAC

Caratteristiche



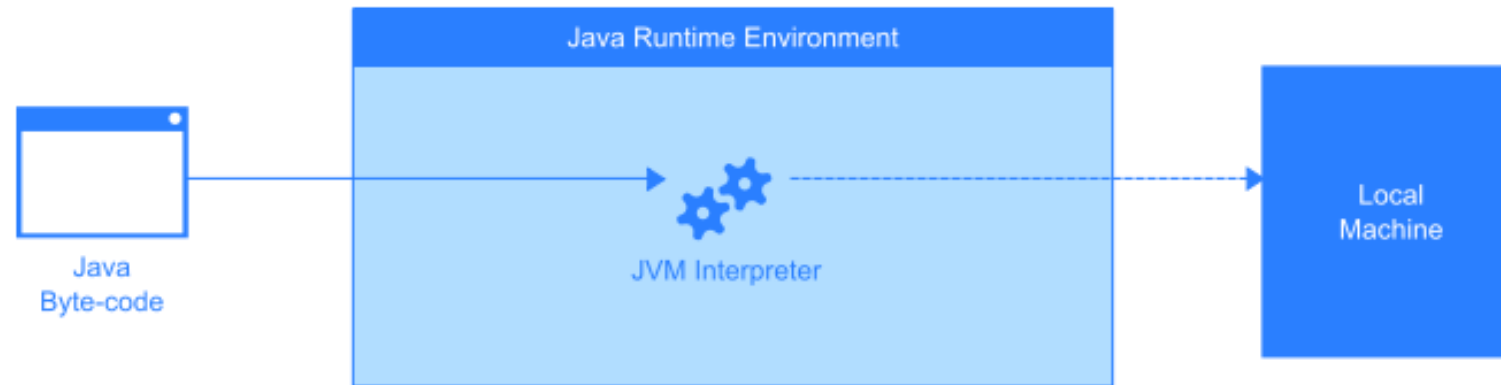
Architettura Java Virtual Machine (JVM)



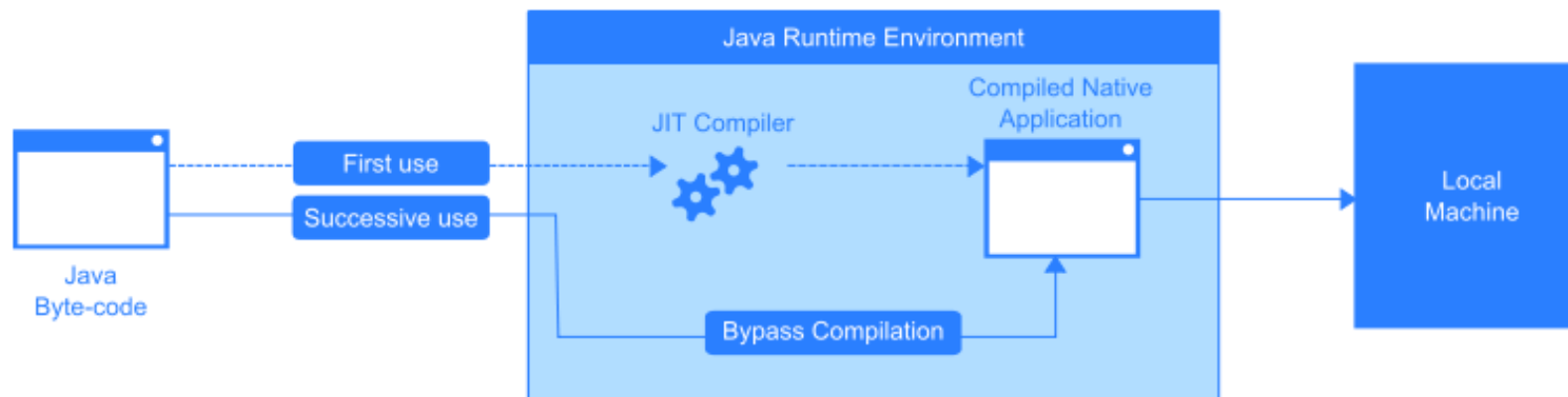
Caracteristici

➤ Execution engine

- **Interpreter:** interpretează și execută bytecode-ul linie cu linie



- **Compiler Just-in-Time (JIT):** transformă bytecode-ul care se execută frecvent în cod mașină nativ, specific procesorului gazdă





Caracteristici

Limbaj orientat pe obiecte

- Orice program conține cel puțin o clasă
- Nu mai există funcții independente

Simplu

- Au fost eliminate concepte precum: pointeri, supraîncărcarea operatorilor, moștenirea multiplă, structuri/uniuni etc.

Robust

- Management automat al memoriei
- Strong data-typed
- Mecanism standard de tratare a excepțiilor

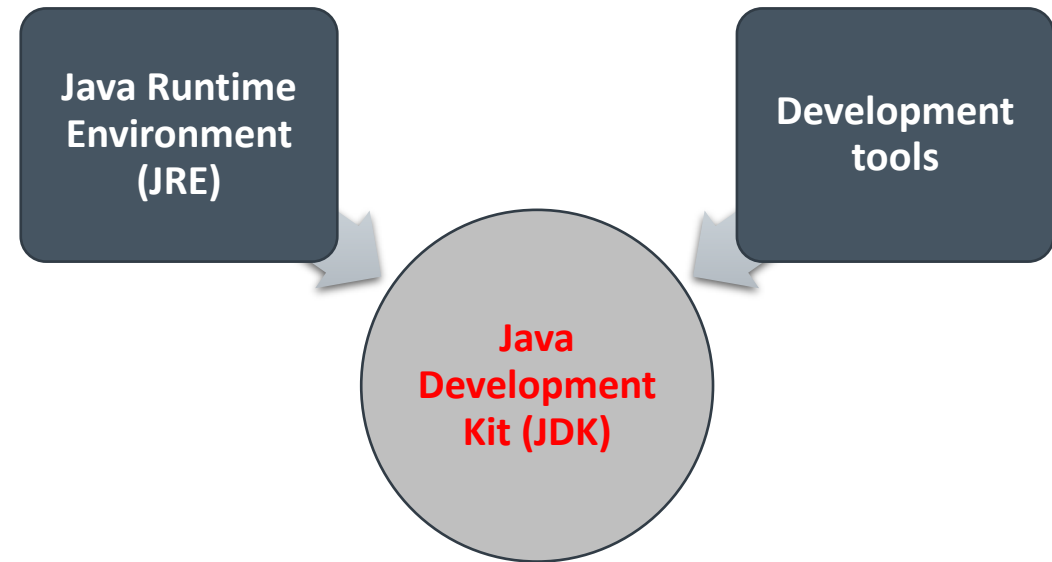
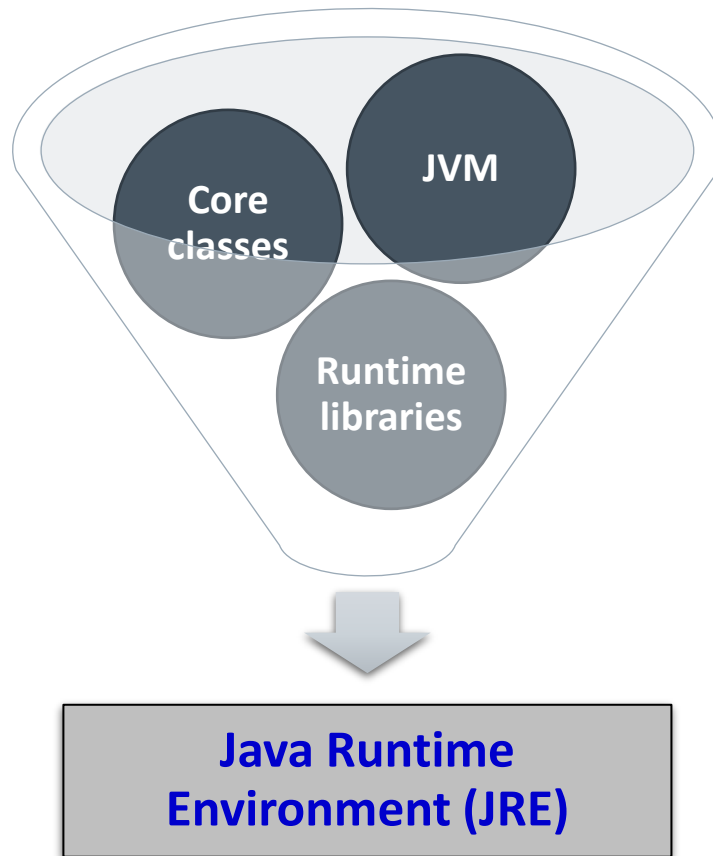
Sigur

- Securitatea platformei
- Sandbox

Permite programarea concurentă și distribuită

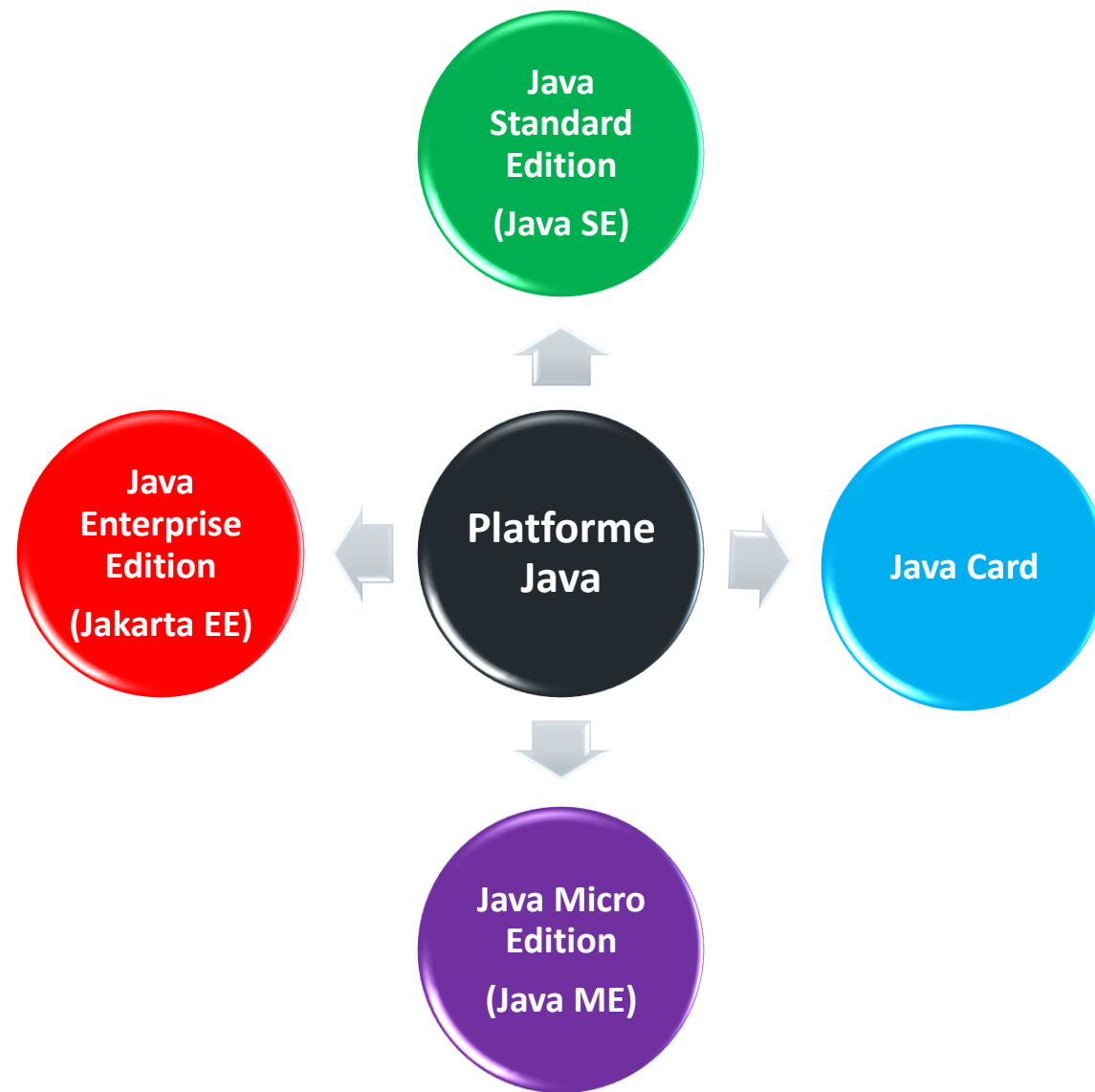
- Suport nativ pentru fire de executare (thread-safe)
- Biblioteci care conțin primitive specifice

Software necesar

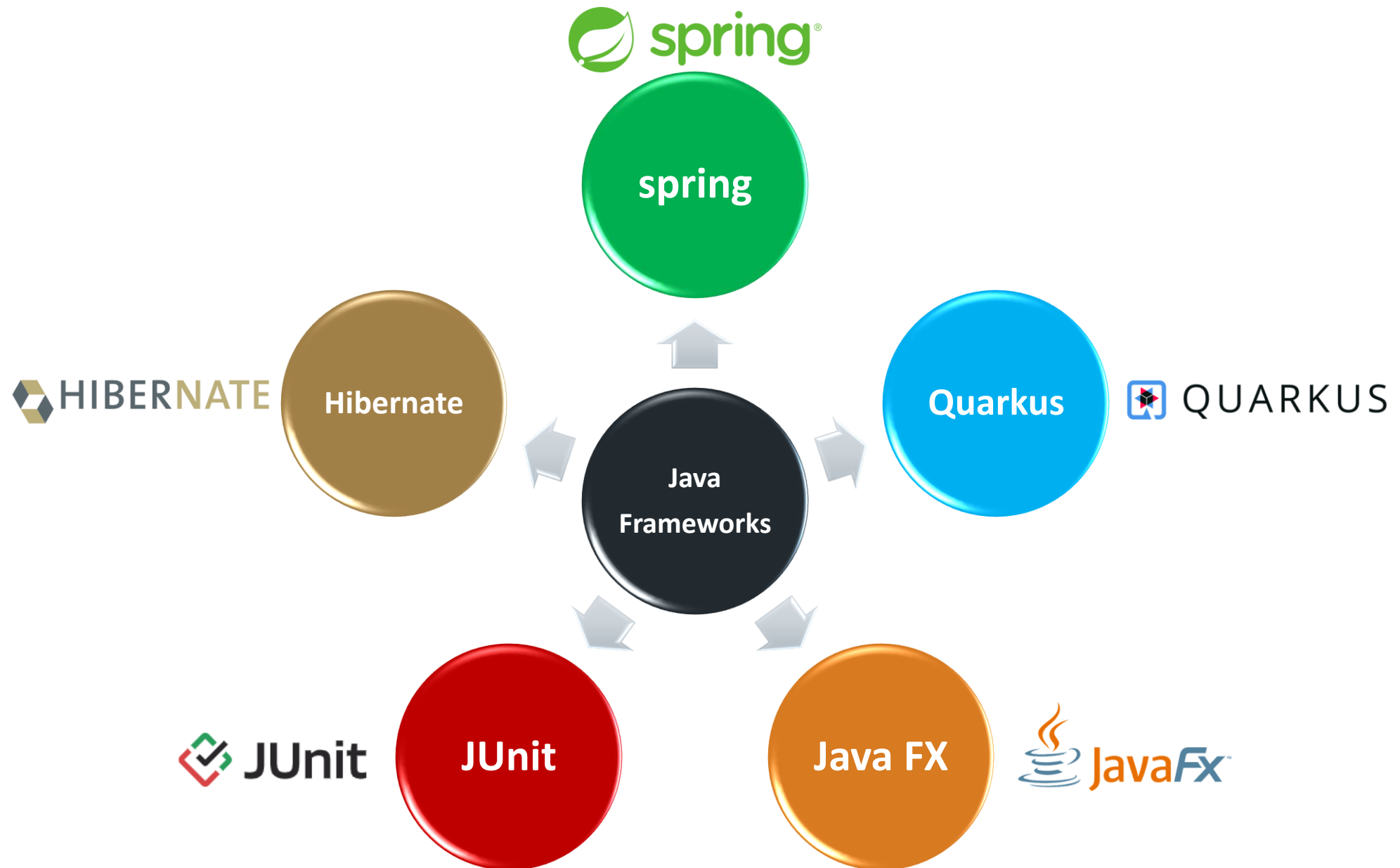




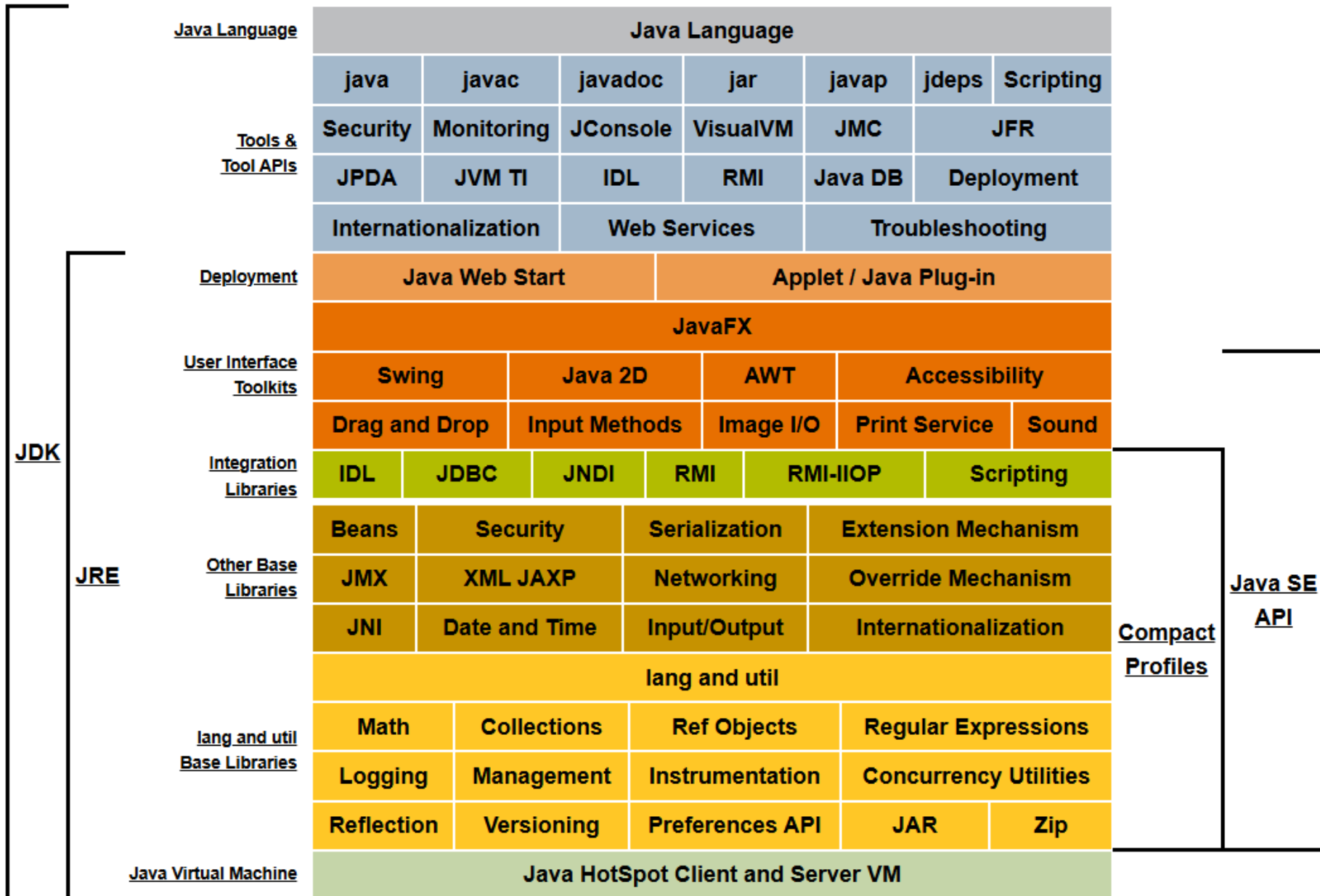
Platforme Java



Framework-uri Java



Structura ediției Java SE 8





- › **Setul de caractere: Unicode (UTF-16)**
- › **Un caracter se reprezintă pe 2 octeți, dar unele caractere Unicode necesită două unități UTF-16 (surrogate pairs).**
- › **Unicode este compatibil cu ASCII pentru primele 128 de caractere.**
- › **Unicode este structurat în blocuri: Basic, Latin, Greek, Arabic, Gothic, Currency, Mathematical, Arrows, Musical etc.**
- › **<http://www.unicode.org>**



➤ Întregi

- pot fi scriși în baza 10, baza 2 (prefix **0b** sau **0B**), baza 8 (prefix **0**) sau baza 16 (prefix **0x** sau **0X**)
- implicit se reprezintă pe 4 octeți (**int**)
- pentru **long** (8 octeți) se adaugă sufixul **L** sau **l**

➤ Numere cu virgulă mobilă

- implicit se reprezintă pe 8 octeți (**double**)
- pentru **float** se adaugă sufixul **F** sau **f**

➤ Boolean: **true** sau **false**

➤ Caractere: **'A'**, număr întreg, **'\u0041'** (format unicode baza 16)

➤ Șiruri de caractere: **"Test"**

➤ **null**

Cuvinte cheie



Java Keywords

<code>abstract</code>	<code>assert</code>	<code>boolean</code>	<code>break</code>	<code>byte</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>class</code>	<code>continue</code>
<code>default</code>	<code>do</code>	<code>double</code>	<code>else</code>	<code>enum</code>
<code>extends</code>	<code>final</code>	<code>finally</code>	<code>float</code>	<code>for</code>
<code>if</code>	<code>implements</code>	<code>import</code>	<code>instanceof</code>	<code>int</code>
<code>interface</code>	<code>long</code>	<code>native</code>	<code>new</code>	<code>package</code>
<code>private</code>	<code>protected</code>	<code>public</code>	<code>return</code>	<code>short</code>
<code>static</code>	<code>strictfp</code>	<code>super</code>	<code>switch</code>	<code>synchronized</code>
<code>this</code>	<code>throw</code>	<code>throws</code>	<code>transient</code>	<code>try</code>
<code>void</code>	<code>volatile</code>	<code>while</code>		

Keywords that are not currently used

<code>const</code>	<code>goto</code>
--------------------	-------------------



TIPURI DE DATE

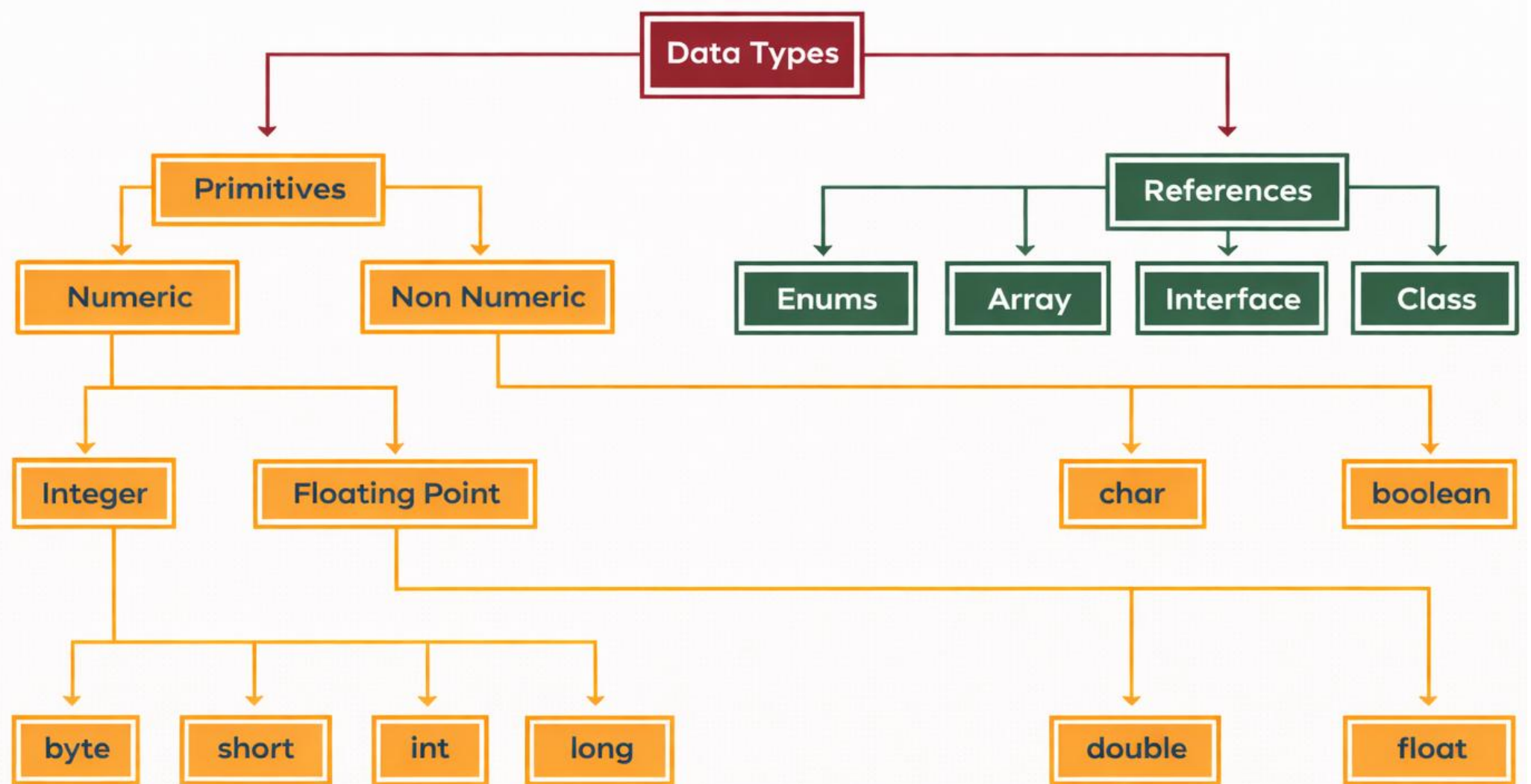
1. Tipurile primitive:

- sunt alocate în zona de memorie de tip stivă
- tipuri numerice (număr întreg, număr "real"), tipul char și tipul boolean

2. Tipuri de date de referință

- sunt obiecte alocate dinamic prin operatorul new in zona de memorie HEAP
- tablouri, clase, interfețe și enumerări

Tipuri de date



Tipuri de date primitive

- Fiecare tip de dată primitiv are asociată o clasă corespunzătoare (**wrapper**), care permite transformarea unei variabile de tip primitiv într-un obiect.

Primitive Types					
Type Name	Wrapper class	Value	Range	Size	Default Value
byte	java.lang.Byte	integer	-128 through +127	8-bit (1-byte)	0
short	java.lang.Short	integer	-32,768 through +32,767	16-bit (2-byte)	0
int	java.lang.Integer	integer	-2,147,483,648 through +2,147,483,647	32-bit (4-byte)	0
long	java.lang.Long	integer	-9,223,372,036,854,775,808 through +9,223,372,036,854,775,807	64-bit (8-byte)	0
float	java.lang.Float	floating point number	$\pm 1.401298\text{E}-45$ through $\pm 3.402823\text{E}+38$	32-bit (4-byte)	0.0
double	java.lang.Double	floating point number	$\pm 4.94065645841246\text{E}-324$ through $\pm 1.79769313486232\text{E}+308$	64-bit (8-byte)	0.0
boolean	java.lang.Boolean	Boolean	true or false	8-bit (1-byte)	false
char	java.lang.Character	UTF-16 code unit (BMP character or a part of a surrogate pair)	'\u0000' through '\uFFFF'	16-bit (2-byte)	'\u0000'

Clase înfășurătoare



Câmpuri statice

- MIN_VALUE
- MAX_VALUE
- SIZE (biți)
- BYTES (octeți)

Constructori

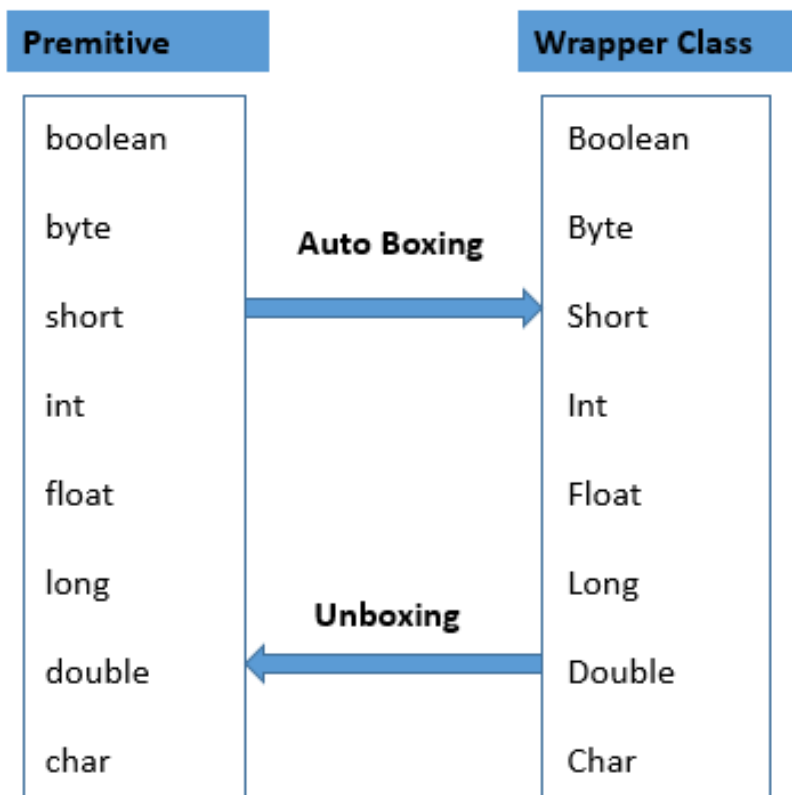
- Cu argument de tip primitiv
- Cu argument de tip String

Metode

- tipValue()
- compare(tip x, tip y)
- compareTo(Tip ob)
- parseTip(String s)
- toString()
- toString(tip x)
- valueOf(tip x)
- valueOf(String s)



Autoboxing/unboxing



```
class Main {  
    public static void main(String [] args) {  
        // Boxing  
        Integer a = 2;  
  
        // UnBoxing  
        int s = 5 + a;  
    }  
}
```



Declararea variabilelor

- La declararea variabilelor de tip primitiv nu se atribuie implicit o valoare reziduală!
- Variabilele locale trebuie să fie inițializate înainte de a fi utilizate, iar datele membre unei clase sunt inițializate implicit cu valori nule de tip!

Operatori



Precedence	Operator	Operand type	Description
1	++,	Arithmetic	Increment and decrement
1	+, -	Arithmetic	Unary plus and minus
1	~	Integral	Bitwise complement
1	!	Boolean	Logical complement
1	(<i>type</i>)	Any	Cast
2	*, /, %	Arithmetic	Multiplication, division, remainder
3	+, -	Arithmetic	Addition and subtraction
3	+	String	String concatenation
4	<<	Integral	Left shift
4	>>	Integral	Right shift with sign extension
4	>>>	Integral	Right shift with no extension
5	<, <=, >, >=	Arithmetic	Numeric comparison
5	instanceof	Object	Type comparison
6	==, !=	Primitive	Equality and inequality of value
6	==, !=	Object	Equality and inequality of reference
7	&	Integral	Bitwise AND
7	&	Boolean	Boolean AND
8	^	Integral	Bitwise XOR
8	^	Boolean	Boolean XOR
9		Integral	Bitwise OR
9		Boolean	Boolean OR
10	&&	Boolean	Conditional AND
11		Boolean	Conditional OR
12	?:	N/A	Conditional ternary operator
13	=	Any	Assignment



Operatori

- Operatorii * și & nu au semnificații specifice pointerilor!
- Nu există operatorul `sizeof`, dimensiunea în octeți pe care se reprezintă tipurile de date primitive se poate determina prin apelul metodei `BYTES` încapsulată în clasele wrapper.
- Operatorul binar `instanceof` testează dacă un obiect este o instanță a unei clase:

`bool` numeObiect `instanceof` NumeClasa

- Operatorii logici `&&` (and), respectiv `||` (or) sunt furnizați și în varianta cu testare fără scurtcircuitare, respectiv `&` și `|`.



Instrucțiuni

Decision-making

- if-then
- if-then-else
- switch

Looping

- while
- do-while
- for

Branching

- break
- continue
- return



Observații

- Pentru instrucțiunea for există, pe lângă forma clasică (loop for), și forma **enhanced for**:

```
int[] numere = {1,2,3,4,5,6,7,8,9,10};  
for (int x : numere)  
    System.out.println(x);
```

Aplicații stand alone



- Sunt programe care pot fi executate în afara contextului unui browser Web.
- Conțin cel puțin o clasă, numită *clasă de bază*, declarată public, **iar denumirea acesteia coincide cu numele fișierului.**
- În cadrul clasei publice trebuie încapsulata funcția principală `main` cu sintaxa :

```
public static void main (String args[])
```



Etapele realizării unei aplicații Java

1. Scrierea codului sursă

- Se poate utiliza orice editor obișnuit sau un mediu specializat (NetBeans, Eclipse etc.)

```
class Test
{
    public static void main( String args[])
    {
        System.out.println("Hello world!");
    }
}
```



2. Salvarea fișierelor sursă

- se va face în fișiere care au obligatoriu extensia **.java**
- denumirea fișierului este aceeași cu cea a clasei publice

3. Compilarea aplicației

- pentru compilare se utilizează compilatorul **javac**:

javac Test.java

4. Rularea aplicației

- se realizează folosind interpretorul **java**:

java Test

Pachete de clase



➤ **Pachet** = colecție de clase și interfețe salvate în același director.

➤ **Utilitatea pachetelor:**

- organizarea fișierelor sursă
- găsirea și utilizarea mai ușoară a claselor
- evitarea conflictelor de nume
- controlul accesului

➤ **Pachete standard Java:** `java.lang`, `java.io`, `java.awt`, `java.math`, `java.util`, `java.sql`, `java.net`, `java.security`



Utilizarea pachetelor de clase

1. Specificarea numelui complet

java.numPachet.NumeClasa // pentru pachete standard

numPachet.NumeClasa // pentru pachete definite de programator

Exemplu:

- **Scanner** - numele clasei care permite citirea formatată
- **java.util** - pachetul din care face parte
- **java.util.Scanner** - numele complet al clasei

```
java.util.Scanner in=new java.util.Scanner(System.in);
```



Utilizarea pachetelor de clase

2. Importul unei clase dintr-un pachet

```
import java.numPachet.numClasa;  
import numPachet.numClasa;
```

Exemplu:

```
import java.util.Scanner;  
Scanner in=new Scanner(System.in);
```

➤ Se specifică clauza import pentru fiecare clasa importată!

```
import java.util.Scanner;  
import java.util.Collections;
```



Utilizarea pachetelor de clase

3. Importul întregului pachet

```
import numePachet.*;
```

Exemplu:

```
import java.awt.*;  
Button b1=new Button("Text");  
TextField t1=new TextField("Java");
```

- Se poate importa orice pachet de clase care nu este standard!!!

```
import nume_pachet.*;  
import nume_pachet.nume_clasa;
```

- Pachetul java.lang se importă implicit în orice sursă Java!!!



Citirea datelor de tip primitiv de la tastatură

➤ Clasa `Scanner` din pachetul `java.util` conține metode pentru citirea formatată de la tastatură.

➤ Crearea unui obiect de tipul `Scanner` pentru citirea de la tastatură:

```
Scanner sc = new Scanner(System.in);
```

➤ Metode pentru citire formatată:

- `nextInt()` – citirea unui număr întreg
- `nextDouble()` – citirea unui număr real
- `next()` – citirea unui șir de caractere fără spații
- `nextLine()` – citirea unui șir de caractere cu spații